DBMS Answer Key

Model answers to the ranked question bank (CEC305, D Y Patil University)

How to use: topics follow the same rank order as the question bank (most repeated first). Each topic lists which papers asked it, then gives an exam-ready answer covering every variation of the question that appeared. Answers are based on your lecture notes (Weeks 1 to 13).

Notes on sources. (1) The Week 13 slides on lock-based protocols contain only pictures, and the notes have no slides on cursors or 5NF, so those answers use standard textbook theory. (2) Week 11 was not among the files you uploaded. (3) Where a question is ambiguous (for example the ALLSTUDENT table has no age column), the assumption is stated in the answer. SQL is written in standard/Oracle style; MySQL differences are mentioned where they matter.

Paper codes: **A** = Jun 2026 CEC305 (60 marks), **B** = Jun 2026 2413CEC3T2 (100 marks), **C** = Dec 2023 CEC305, **D** = Dec 2023 RAIT.

#	Topic	Papers
1	Normal forms and normalization	A B C D
2	ER to relational schema mapping	A B C D
3	Relational algebra operations	A B C D
4	Functional dependency	A B C D
5	Conflict and view serializability	A B C D
6	SQL query-writing problems	A B C D
7	Triggers	A B C D
8	ER diagram design (University, Hospital EER, Company, Car insurance)	A B C D
9	DDL and DML commands	A B C D
10	Transactions and ACID	B C D
11	Generalization and specialization	B C D
12	SQL constraints	B C D
13	Lock-based protocols, 2PL, deadlock	A D
14	SQL joins	A B
15	Stored procedures	A B
16	DBMS architecture	A C
17	SQL vs PL/SQL	B C
18	Types of ER attributes	B C
19	SQL aggregate functions	B C
20	Recovery system	A
21	Transaction states	B
22	Cursors and functions in PL/SQL	B
23	Total and partial participation	B
24	Super key and candidate key	B
25	DBA responsibilities	D

1. Normal Forms and Normalization

Asked in: A Q1c · B Q1d, Q2d, Q4b · C Q1c, Q3b · D Q2A-iii

Normalization and why it is needed

Normalization is the process of organizing a database into smaller, related tables to reduce redundancy and dependency and to improve data integrity. A poorly designed (flat) table suffers from three anomalies:

- **Insertion anomaly:** a new employee who is not yet assigned to a department cannot be stored if the department column does not allow NULL.
- **Deletion anomaly:** deleting all rows of a closed department also deletes information about employees who belonged only to it.
- **Update anomaly:** an employee stored in two rows (two departments) needs his address changed in both rows, otherwise the data becomes inconsistent.

First Normal Form (1NF)

Definition: a relation is in 1NF if every attribute holds only atomic (indivisible) values, i.e. there are no multivalued or composite attributes and no repeating groups. **Conditions:** (a) each cell holds a single value; (b) each column holds values of the same type; (c) each column has a unique name; (d) each row is unique (a primary key exists); (e) order of rows and columns does not matter.

Example. STUDENT is not in 1NF because one cell stores two phone numbers:

Roll	Name	Phone
1	Amit	98201, 98202
2	Neha	99999

Conversion to 1NF (Approach 2 in the notes, the best one): split the multivalued attribute into its own table, carrying the key.

STUDENT(Roll, Name) and **STUDENT_PHONE(Roll, Phone)**:

Roll	Name
1	Amit
2	Neha

Roll	Phone
1	98201
1	98202
2	99999

Other approaches: make (Roll, Phone) a composite key (causes redundancy of Name), or add columns Phone1, Phone2 (wastes space with many NULLs). Now every cell is atomic, so both tables satisfy the 1NF conditions.

Second Normal Form (2NF)

Definition: a relation is in 2NF if it is in 1NF and has no *partial dependency*, meaning no non-prime attribute depends on only a part (proper subset) of a candidate key.

Example. TEACHER(TEACHER_ID, SUBJECT, TEACHER_AGE) with candidate key {TEACHER_ID, SUBJECT}. TEACHER_AGE depends on TEACHER_ID alone, a partial dependency, so it violates 2NF. Decompose:

```
TEACHER_DETAIL(TEACHER_ID, TEACHER_AGE)      key: TEACHER_ID
TEACHER_SUBJECT(TEACHER_ID, SUBJECT)          key: (TEACHER_ID, SUBJECT)
```

Third Normal Form (3NF)

Definition: a relation is in 3NF if it is in 2NF and has no transitive dependency of a non-prime attribute on a key. Formally, for every non-trivial FD X → A, either X is a super key or A is a prime attribute.

Example. EMPLOYEE(emp_id, emp_name, emp_zip, emp_state, emp_city, emp_district). Here emp_id → emp_zip and emp_zip → {state, city, district}, so state, city and district are transitively dependent on emp_id. Decompose:

```
EMPLOYEE(emp_id, emp_name, emp_zip)
EMPLOYEE_ZIP(emp_zip, emp_state, emp_city, emp_district)
```

BCNF (for completeness)

A relation is in BCNF if for every non-trivial FD X → Y, X is a super key. Example: R(student, teacher, subject) with teacher → subject is in 3NF but not BCNF (teacher is not a key); decompose into R1(teacher, subject) and R2(student, teacher).

Fifth Normal Form (5NF), with student database example (C Q1c)

A relation is in **5NF** (project-join normal form) if it is in 4NF and every join dependency in it is implied by its candidate keys; i.e. it cannot be split further into smaller relations whose natural join recreates it without loss, except through keys. 5NF removes redundancy that arises from *cyclic* constraints.

Student database example. R(Student, Course, Instructor) records which student takes which course from which instructor. Suppose the business rule is cyclic: *if a student takes a course, the instructor teaches that course, and the student is taught by that instructor, then the student takes that course from that instructor.* The only candidate key is all three attributes, so the join dependency is not implied by a key. Decompose into three relations:

```
R1(Student, Course)      R2(Course, Instructor)      R3(Student, Instructor)
# R = R1 JOIN R2 JOIN R3  (lossless), so R1, R2, R3 are in 5NF
```

Lossy and lossless decomposition (B Q2d)

A decomposition of R into R1 and R2 is **lossless** if R1 ⋈ R2 = R exactly; it is **lossy** if the join produces spurious (extra) tuples or loses tuples. Lossless condition: the common attributes R1 ∩ R2 must be a key of R1 or R2.

Example of a lossy decomposition. R(A, B, C) = {(1,x,p), (2,x,q)}. Decompose into R1(A,B) = {(1,x),(2,x)} and R2(B,C) = {(x,p),(x,q)}. The common attribute B is a key of neither. R1 ⋈ R2 = {(1,x,p),(1,x,q),(2,x,p),(2,x,q)}: 4 tuples instead of 2, with the spurious tuples (1,x,q) and (2,x,p). Information is lost because we can no longer tell which tuples were original. Decomposing on A instead (R1(A,B), R2(A,C)) is lossless since A is a key.

Problem 1 (B Q4b): Faculty table, identify normal form and normalize

F_id	F_address	Course_id	Course_name
101	Delhi	C1	MCA
101	Delhi	C2	MBA
102	Noida	C1	MCA
102	Noida	C2	MBA

- **Non-trivial FD:** an FD X → Y where Y is not a subset of X. Here F_id → F_address and Course_id → Course_name are non-trivial.
- **Candidate key:** (F_id, Course_id). All attributes are atomic, so the table is in **1NF**.
- F_address depends only on F_id and Course_name only on Course_id: **partial dependencies**, so the table is **not in 2NF**.
- **Convert to 2NF** (the results are also in 3NF/BCNF):

```
FACULTY(F_id, F_address)              key: F_id
COURSE(Course_id, Course_name)          key: Course_id
TEACHES(F_id, Course_id)                key: (F_id, Course_id)
```

Problem 2 (C Q3b): R(A,B,C,D), FD = {AB → CD, B → C}. Is R in 2NF?

- Closure: (AB)+ = {A,B,C,D}, so **AB is the candidate key** (no smaller set works: A+ = {A}, B+ = {B,C}). Prime attributes: A, B. Non-prime: C, D.
- B → C: the non-prime attribute C depends on B, which is a proper subset of the key AB. This is a **partial dependency**, so R is **not in 2NF** (it is in 1NF).
- **Convert to 2NF:** remove C from R and place it with B.

```
R1(B, C)        key: B       (B → C)
R2(A, B, D)     key: AB      (AB → D, D depends on the whole key)
```

The decomposition is lossless (the common attribute B is the key of R1) and dependency-preserving (AB → CD is recovered from AB → D together with B → C). Both R1 and R2 are in 2NF.

2. ER to Relational Schema Mapping

Asked in: A Q2b · B Q2b, Q5a · C Q3a · D Q2A-i, Q3ii-b

Rules / steps for mapping an ER (or EER) diagram

Step	ER construct	Relational mapping
1	Strong entity set	Create a table; attributes become columns; the key attribute becomes the primary key.
2	Composite attribute	Do not store the composite itself; its component attributes become columns (Salary → Basic, DA, HRA).
3	Multivalued attribute	Create a separate table with the entity's key plus the multivalued attribute; PK = (key, attribute). Example PERSON_PHONE(Emp_ID, Phone).
4	Derived attribute	Not stored (computed when needed, e.g. Age from DOB).
5	Weak entity set	Table containing its partial key plus the owner's primary key as a foreign key. PK = (owner key, partial key).
6	1:1 relationship	Add the key of one side as a foreign key on the other. If one entity has total participation, merge into one table (or put the FK on the total side). If both are partial, keep two tables (or three with a relationship table) because NULLs appear.
7	1:N relationship	Put the primary key of the '1' side as a foreign key in the 'N' side table. Relationship attributes also move to the N side.
8	M:N relationship	Create a new table holding the primary keys of both entities (as FKs) plus relationship attributes. PK = combination of both keys.
9	Generalization / specialization	Table for the superclass; one table per subclass with the superclass key as PK and FK (or one single table with a type column).

One-to-one and many-to-many conversion with examples (B Q2b)

One-to-one: PERSON(Per_Id, Name) – HAS – PASSPORT(Pass_No). A person has at most one passport; every passport belongs to a person. With total participation of PASSPORT, merge: **PERSON(Per_Id, Name, Pass_No)**. If both sides are partial (e.g. MALE – MARRIES – FEMALE), merging would give NULLs, so keep MALE(M_Id, ..., F_Id) and FEMALE(F_Id, ...).

Many-to-many: STUDENT(S_Id, Name) – ENROLLS – COURSE(C_Id, Title). Create a third table: **ENROLLS(S_Id, C_Id, Grade)** with PK (S_Id, C_Id), where S_Id and C_Id are foreign keys. This avoids repeating groups in either entity table.

Hospital EER, Company ER and Car Insurance ER converted to schemas

(The diagrams are in Topic 8. The schemas below follow them.)

Hospital EER (B Q5a):

```
PATIENT(Patient_ID PK, Name, DOB, Ward_No FK)        -- N:1 'admitted to' merged here
PATIENT_PHONE(Patient_ID FK, Phone)                  PK (Patient_ID, Phone)   -- multivalued
DOCTOR(Doctor_ID PK, Name, Specialty)
SURGEON(Doctor_ID PK/FK, ...)   PHYSICIAN(Doctor_ID PK/FK, ...)              -- specialization
WARD(Ward_No PK, Type)
TREATED_BY(Patient_ID FK, Doctor_ID FK)              PK (Patient_ID, Doctor_ID)  -- M:N
TEST_LOG(Patient_ID FK, Test_No, Test_Type, Result, Test_Date)
                                                     PK (Patient_ID, Test_No)  -- weak entity
```

Company (C Q3a):

```
EMPLOYEE(Emp_ID PK, Name, Salary, Address, Dept_No FK)          -- 'works for' N:1
DEPARTMENT(Dept_No PK, Dept_Name)
DEPT_LOCATIONS(Dept_No FK, Location)                            -- multivalued
PROJECT(Proj_No PK, Proj_Name, Location, Dept_No FK)            -- 'controls' 1:N
WORKS_ON(Emp_ID FK, Proj_No FK, Hours)                          PK (Emp_ID, Proj_No)
DEPENDENT(Emp_ID FK, Dep_Name, Relation, Age)                   PK (Emp_ID, Dep_Name)
```

Car insurance (D Q3ii-b):

```
PERSON(Driver_ID PK, Name, Address)
CAR(Licence PK, Model, Year, Driver_ID FK)                      -- owns, 1:N
ACCIDENT(Report_No PK, Acc_Date, Location)
PARTICIPATED(Licence FK, Report_No FK, Damage_Amount)           PK (Licence, Report_No)
```

3. Relational Algebra Operations

Asked in: A Q1b, Q3b · B Q3b, Q5b · C Q1b · D Q3i

Relational algebra is a **procedural** query language: queries are written as step-by-step expressions using operators on relations. Operators are **unary** (σ, π, ρ) or **binary** (∪, −, ∩, ×, ⋈, ÷). Sample relations used below:

STUDENT(Roll, Name, Age, City)

Roll	Name	Age	City
1	Amit	19	Delhi
2	Neha	21	Mumbai
3	Raj	18	Delhi
4	Sara	22	Pune

Unary operations

Operation	Syntax	Meaning and example
Selection σ	$\sigma_{condition}(R)$	Selects **rows** (tuples) satisfying the condition. $\sigma_{Age>18}$(STUDENT) gives Amit, Neha, Sara.
Projection π	$\pi_{A1,A2}(R)$	Selects **columns** and removes duplicate rows. $\pi_{Roll,Name}$(STUDENT) gives (1,Amit), (2,Neha), (3,Raj), (4,Sara). π_{City}(STUDENT) gives only 3 rows: Delhi, Mumbai, Pune (duplicate Delhi removed).
Rename ρ	ρ(S, R)	Gives a new name to a relation or its attributes. ρ(STUDENT1, STUDENT).

Combined: names of students older than 18: $\pi_{Name}(\sigma_{Age>18}(STUDENT))$ = {Amit, Neha, Sara}. (Select first, then project.) Other unary queries: $\pi_{Name,City}(\sigma_{City='Delhi'}(STUDENT))$; on the bank database, $\sigma_{amount>1200}(loan)$ and $\pi_{loan_number}(\sigma_{amount>1200}(loan))$.

Binary set operations

Union, difference and intersection need **union-compatible** relations: same number of attributes, with matching domains.

Operation	Syntax	Meaning and example
Union ∪	R ∪ S	Tuples in R or S or both; duplicates appear once. STUDENT ∪ EMPLOYEE gives all persons who are students or employees.
Set difference −	R − S	Tuples in R but not in S. STUDENT − EMPLOYEE gives students who are not employees.
Intersection ∩	R ∩ S	Tuples in both R and S. $\pi_{customer_name}$(borrower) ∩ $\pi_{customer_name}$(depositor) gives customers with both a loan and an account. (R ∩ S = R − (R − S)).
Cartesian product ×	R × S	Every tuple of R paired with every tuple of S. If R has m tuples and S has n, the result has m × n tuples and the attributes of both. Normally followed by σ to make a meaningful join.

Join operations

A join combines related tuples of two relations: it is a Cartesian product followed by selection (and projection). With STUDENT(S_id, Name, C_id) and COURSE(C_id, C_name):

- **Natural join (R ⋈ S):** joins on all attributes with the same name and keeps one copy of the common column. STUDENT ⋈ COURSE.
- **Theta join ($R \bowtie_\theta S$):** join on any condition with a comparison operator (<, >, =, ...). **Equi join** is the special case where θ uses only '='.
- **Outer joins:** keep unmatched tuples padded with NULL: left outer (all of R), right outer (all of S), full outer (all of both).

Division (÷) [C Q1b(ii)]

R(A, B) ÷ S(B) gives the values of A that are associated in R with **every** tuple of S. Example: TAKES(Student, Course) ÷ REQUIRED(Course) lists students who have taken *all* required courses.

Student	Course
Amit	DBMS
Amit	OS
Neha	DBMS
Raj	OS

(TAKES relation above.) With REQUIRED = {DBMS, OS}: TAKES ÷ REQUIRED = {Amit}, because only Amit is paired with both courses. Division can be expressed as $\pi_{Student}(TAKES) - \pi_{Student}((\pi_{Student}(TAKES) \times REQUIRED) - TAKES)$.

Further example queries (bank database)

```
Customers with a loan at Perryridge branch:
  π customer_name ( σ branch_name='Perryridge' ( σ borrower.loan_number = loan.loan_number (borrower × loan) ) )
Same, but without an account at any branch:
  π customer_name(... above ...)  −  π customer_name(depositor)
Customers with a loan, an account, or both:
  π customer_name(borrower) ∪ π customer_name(depositor)
```

4. Functional Dependency

Asked in: A Q2c · B Q3e, Q4b · C Q3b · D Q1ii

Definition

A **functional dependency (FD)** X → Y states that for each value of X there is exactly one associated value of Y; X is the **determinant** and Y the **dependent**. If two tuples agree on X they must agree on Y. Example: in STUDENT(roll_no, name, dept_name, dept_building) we have roll_no → name and dept_name → dept_building are valid, but name → dept_name is invalid (two students with the same name can be in different departments), and dept_building → dept_name is invalid (ME and EC share building B2).

Armstrong's axioms

- **Reflexivity:** if Y ⊆ X then X → Y. Example: {roll_no, name} → name.
- **Augmentation:** if X → Y then XZ → YZ.
- **Transitivity:** if X → Y and Y → Z then X → Z. Example: roll_no → dept_name and dept_name → dept_building give roll_no → dept_building.

Types of functional dependency

Type	Meaning	Example
Trivial	Y is a subset of X	{roll_no, name} → name
Non-trivial	Y is not a subset of X	roll_no → name
Transitive	X → Y and Y → Z with Y not a key, so X → Z indirectly	enrol_no → dept, dept → building_no
Full	Y depends on the whole key X and on no subset of it	(student_id, subject_id) → marks
Partial	A non-key attribute depends on only part of a composite key	subject_id → teacher in key (student_id, subject_id)
Multivalued (notes)	One determinant, dependents independent of each other	roll_no → {name, age}

Role in normalization and design

- FDs identify **keys** (a set whose closure is all attributes is a key), which is the basis of every normal form.
- **1NF/2NF/3NF/BCNF** are defined by FDs: 2NF forbids *partial* FDs, 3NF forbids *transitive* FDs, BCNF requires every determinant to be a super key.
- They guide **decomposition** to remove anomalies while keeping the decomposition lossless and dependency-preserving.
- They enforce integrity constraints and help in query optimization (knowing how tables connect).

Closure of attributes (useful for FD problems)

The closure X+ is the set of all attributes determined by X. Method: start with X, repeatedly add attributes of any FD whose left side is already in the set. Example: R(A,B,C,D,E), F = {A → B, B → C, D → E}: (AD)+ = {A,D} → {A,D,B} → {A,D,B,C} → {A,D,B,C,E} = all attributes, so AD is a candidate key.

Related worked problems. The Faculty/Course table (B Q4b) and R(A,B,C,D) with FD = {AB → CD, B → C} (C Q3b) are solved in Topic 1, because they test FDs together with normal forms.

5. Conflict and View Serializability

Asked in: A Q3c · B Q5c · C Q2f · D Q2B-i

Basic concepts

- A **schedule** is an ordering of the operations of several transactions. A **serial schedule** runs one transaction completely before the next; a **non-serial** (interleaved) schedule mixes them. A non-serial schedule is **serializable** if its result equals that of some serial schedule, so it keeps the database consistent.
- Notation: R(X) reads item X, W(X) writes X; subscripts give the transaction ($R_1(A)$ = T1 reads A).

Conflicting operations

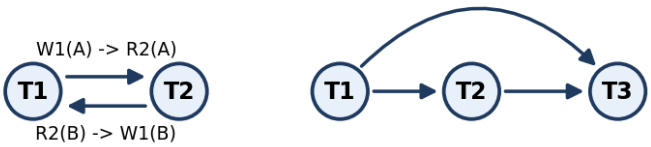
Two operations conflict if (1) they belong to **different transactions**, (2) they access the **same data item**, and (3) **at least one is a write**. So the conflicting pairs are R–W, W–R and W–W; R–R never conflicts.

Conflict serializability and the precedence graph

A schedule is **conflict serializable** if it can be turned into a serial schedule by swapping adjacent *non-conflicting* operations. Test with a **precedence (serialization) graph**: one node per transaction; draw an edge $T_i \rightarrow T_j$ if an operation of T_i conflicts with a later operation of T_j. **If the graph has no cycle, the schedule is conflict serializable**, and a topological order of the graph gives the equivalent serial schedule. A cycle means it is not.

Example 1 (not conflict serializable): S1 = $R_1(A)$, $W_1(A)$, $R_2(A)$, $R_2(B)$, $W_1(B)$. $W_1(A)$ before $R_2(A)$ gives T1 → T2; $R_2(B)$ before $W_1(B)$ gives T2 → T1. The graph has a cycle T1 ⇄ T2, so S1 is **not** conflict serializable.

Example 2 (serializable): a graph with edges T1 → T2, T2 → T3, T1 → T3 has no cycle. T1 has no incoming edge so it runs first, then T2, then T3: equivalent serial order T1 → T2 → T3.



Left: cyclic graph of Example 1. Right: acyclic graph of Example 2.

View serializability

A schedule S is **view serializable** if it is **view equivalent** to some serial schedule S'. Two schedules are view equivalent if:

- **Initial read:** if T_i reads the initial value of a data item in S, it does so in S' too.
- **Read-from (updated read):** if T_i reads a value written by T_j in S, it reads from T_j in S' too.
- **Final write:** the transaction that performs the last write of each item in S also does it in S'.

Every conflict-serializable schedule is view serializable, but not the reverse. A view-serializable schedule that is not conflict serializable must contain a **blind write** (a write without a prior read of that item). Procedure: check conflict serializability first; if it fails, look for blind writes; if there are none the schedule is not view serializable; otherwise try to find a view-equivalent serial order.

Example 3: S = $R_1(A)$, $W_2(A)$, $W_1(A)$, $W_3(A)$. Precedence graph: $R_1(A) \rightarrow W_2(A)$ gives T1 → T2, and $W_2(A) \rightarrow W_1(A)$ gives T2 → T1: a cycle, so **not** conflict serializable. But T2 and T3 perform blind writes. Compare with the serial schedule T1, T2, T3: T1 reads the initial A (same), nobody reads another's write (same), and T3 does the final write (same). So S is **view serializable**, equivalent to T1 → T2 → T3.

Conflict vs view serializability

Conflict serializability	View serializability
Based on conflicting pairs and swapping non-conflicting operations	Based on view equivalence (initial read, read-from, final write)
Tested in polynomial time with a precedence graph	Testing is NP-complete
Stricter: a subset of view-serializable schedules	Broader; allows schedules with blind writes

6. SQL Query-Writing Problems

Asked in: A Q3a · B Q4a · C Q3c · D Q2B-ii

A Q3a: ALLSTUDENT(ROLLNO, FULLNAME, ADDRESS, COURSE, MARKS)

Assumption: the table as given has no age or date-of-birth column. For part (i) the answer assumes a DOB column (or an AGE column). MARKS is assumed to be out of 100, so 75% means MARKS > 75.

(i) Students older than 19. Age is derived from the date of birth by taking the difference between today's date and DOB in years, then filtering with WHERE.

```
-- If a DOB column exists (MySQL)
SELECT * FROM ALLSTUDENT WHERE TIMESTAMPDIFF(YEAR, DOB, CURDATE()) > 19;
-- Oracle
SELECT * FROM ALLSTUDENT WHERE MONTHS_BETWEEN(SYSDATE, DOB)/12 > 19;
-- If an AGE column exists
SELECT * FROM ALLSTUDENT WHERE AGE > 19;
```

Logic: the date function computes each student's age row by row; WHERE keeps only rows where the condition is true, so students aged 19 or younger are filtered out.

(ii) Names of students with marks above 75%.

```
SELECT FULLNAME FROM ALLSTUDENT WHERE MARKS > 75;
```

Logic: the comparison operator > is applied to the numeric MARKS column in every row; only the FULLNAME column is projected. The comparison is strict, so a student with exactly 75 is excluded (use >= to include).

(iii) Names beginning with 'S'.

```
SELECT FULLNAME FROM ALLSTUDENT WHERE FULLNAME LIKE 'S%';
```

Logic: LIKE does pattern matching; the wildcard % stands for any sequence of characters (including none), so 'S%' means the first letter is S followed by anything. (_ matches exactly one character.)

(iv) Students living in 'Andheri'.

```
SELECT * FROM ALLSTUDENT WHERE ADDRESS LIKE '%Andheri%';
-- or, if ADDRESS holds only the area name:  WHERE ADDRESS = 'Andheri';
```

Logic: an address is usually a long string (flat, street, area), so '%Andheri%' searches for the word anywhere inside it; rows whose address does not contain it are filtered out.

B Q4a: Employee database

Employee(empname, street, city, date_of_joining), Works(empname, company_name, salary), Company(company_name, city), Manages(empname, manager_name)

```
(i)   UPDATE Employee SET city = 'Mumbai' WHERE empname = 'John';

(ii)  SELECT * FROM Employee WHERE EXTRACT(MONTH FROM date_of_joining) = 10;
      -- MySQL: WHERE MONTH(date_of_joining) = 10

(iii) UPDATE Works SET salary = salary * 1.10
      WHERE company_name = 'TCS Corporation';
      -- (assumes "increased by 10%" means give a 10% raise; the schema has no year column)

(iv)  SELECT E.empname
      FROM Employee E, Works W, Company C
      WHERE E.empname = W.empname
        AND W.company_name = C.company_name
        AND E.city = C.city;

(v)   SELECT W.empname
      FROM Works W
      WHERE W.salary > (SELECT AVG(W2.salary) FROM Works W2
                        WHERE W2.company_name = W.company_name);   -- correlated subquery
```

C Q3c: First Bank Corporation

Employee(employee_name, street, city), Works(employee_name, company_name, salary), company(company_name, city), manages(employee_name, manager_name)

```
(i)   SELECT employee_name FROM Works
      WHERE company_name = 'First Bank Corporation';

(ii)  SELECT E.employee_name, E.city
      FROM Employee E, Works W
      WHERE E.employee_name = W.employee_name
        AND W.company_name = 'First Bank Corporation';

(iii) SELECT E.employee_name, E.street, E.city
      FROM Employee E, Works W
      WHERE E.employee_name = W.employee_name
        AND W.company_name = 'First Bank Corporation'
        AND W.salary > 10000;

(iv)  SELECT employee_name FROM Employee
      WHERE employee_name NOT IN (SELECT employee_name FROM Works
                                  WHERE company_name = 'First Bank Corporation');

(v)   SELECT AVG(salary) FROM Works;
```

D Q2B-ii: Employee(Emp_no, Name, Emp_city), Company(Emp_no, Company_name, Salary)

```
(i)   SELECT E.Name, C.Company_name
      FROM Employee E, Company C WHERE E.Emp_no = C.Emp_no;

(ii)  SELECT E.Name, E.Emp_city, C.Company_name, C.Salary
      FROM Employee E, Company C
      WHERE E.Emp_no = C.Emp_no AND C.Salary > 10000;

(iii) SELECT E.Name
      FROM Employee E, Company C
      WHERE E.Emp_no = C.Emp_no AND C.Company_name = 'XYZ';
```

7. Triggers

Asked in: A Q2e · B Q4c · C Q2d · D Q2A-iv

Definition, uses and advantages

A **trigger** is a stored PL/SQL program that is **fired automatically** when a specified event occurs: a DML statement (INSERT, UPDATE, DELETE), a DDL statement (CREATE, ALTER, DROP), or a database event (LOGON, LOGOFF, STARTUP, SHUTDOWN).

- **Uses:** enforce complex constraints that PK/FK/UNIQUE cannot express, record changes (auditing), generate primary key values, prevent invalid transactions, enforce security and referential integrity.
- **Advantages:** automatic enforcement of rules, central monitoring and logging, can stop invalid transactions, produce derived column values.

Types of triggers

Classification	Types
By level	**Row-level** (FOR EACH ROW): fires once for every row affected. **Statement-level**: fires once per SQL statement, however many rows it affects.
By timing	BEFORE, AFTER, INSTEAD OF (on views)
By event	DML trigger, DDL trigger, database-event trigger

Statement-level trigger (D Q2A-iv)

A statement-level trigger has no FOR EACH ROW clause; it executes **exactly once** for the triggering statement, even if the statement changes 0, 1 or 1000 rows, and it cannot use :OLD and :NEW. It is suited to checks on the statement as a whole (who ran it, when, is it allowed now).

```
CREATE OR REPLACE TRIGGER trg_emp_stmt
BEFORE INSERT OR UPDATE OR DELETE ON employee      -- no FOR EACH ROW
BEGIN
  dbms_output.put_line('DML executed on EMPLOYEE at ' || TO_CHAR(SYSDATE,'HH24:MI:SS'));
END;
/
```

General syntax

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE | DELETE} [OR ...] [OF column]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW] [WHEN (condition)]
DECLARE ... BEGIN ... END;
```

Trigger on the employee database (B Q4c, C Q2d)

EMPLOYEE(EID, ENAME, SALARY): show old salary, new salary and the difference on every change.

```
CREATE OR REPLACE TRIGGER salary_difference
BEFORE DELETE OR INSERT OR UPDATE ON employee
FOR EACH ROW
DECLARE
  sal_diff NUMBER;
BEGIN
  sal_diff := :NEW.salary - :OLD.salary;
  dbms_output.put_line('Old salary: '  || :OLD.salary);
  dbms_output.put_line('New salary: '  || :NEW.salary);
  dbms_output.put_line('Difference: '  || sal_diff);
END;
/
UPDATE employee SET salary = 60000 WHERE eid = 1;   -- old 30000, new 60000, difference 30000
```

On INSERT :OLD values are NULL; on DELETE :NEW values are NULL. A trigger is enabled or disabled with `ALTER TRIGGER salary_difference DISABLE;`

Audit trigger for the bank Accounts table (A Q2e)

```
CREATE TABLE audit_log (
  account_id   NUMBER,
  old_balance  NUMBER,
  new_balance  NUMBER,
  action_type  VARCHAR2(10),
  action_time  DATE
);

CREATE OR REPLACE TRIGGER trg_accounts_audit
AFTER INSERT OR UPDATE OR DELETE ON accounts
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO audit_log VALUES (:NEW.account_id, NULL, :NEW.balance, 'INSERT', SYSDATE);
  ELSIF UPDATING THEN
    INSERT INTO audit_log VALUES (:OLD.account_id, :OLD.balance, :NEW.balance, 'UPDATE', SYSDATE);
  ELSIF DELETING THEN
    INSERT INTO audit_log VALUES (:OLD.account_id, :OLD.balance, NULL, 'DELETE', SYSDATE);
  END IF;
END;
/
```

Evaluation (data integrity and consistency):

- Every change to an account is recorded automatically with the **before and after balance**, so the history is traceable and unauthorized or erroneous changes can be detected and reversed.
- It cannot be bypassed by an application or user, because it fires inside the database whatever tool made the change (accountability, security).
- Because it is an AFTER trigger in the same transaction, the audit row commits or rolls back together with the data change, so the log and the table never disagree (consistency).
- **Limitations:** extra write for each change (performance cost), the audit table grows and needs archiving, and the trigger must be protected from being disabled by unauthorized users.

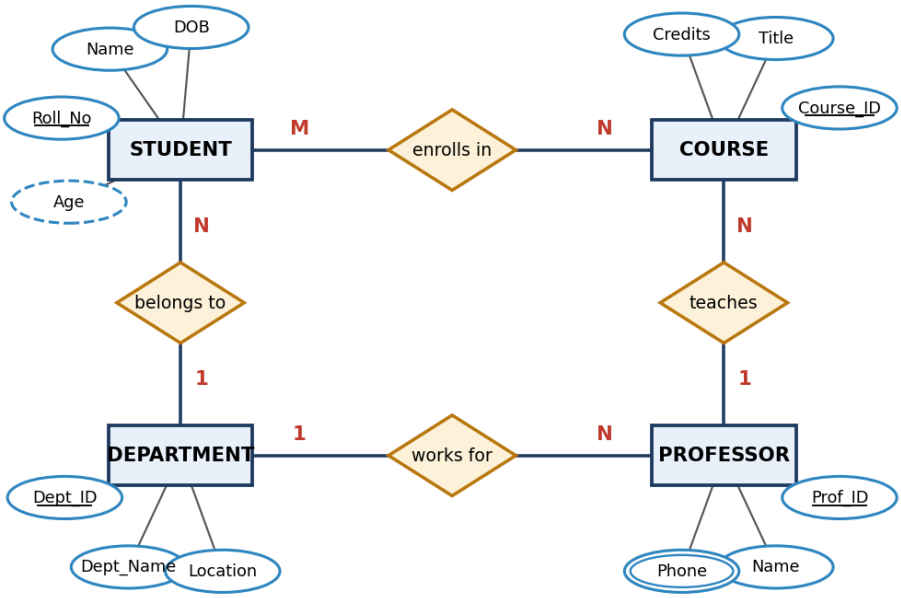
8. ER Diagram Design

Asked in: A Q2a · B Q5a · C Q3a · D Q3ii-a

Notation: rectangle = entity, double rectangle = weak entity, ellipse = attribute (underlined = key, double ellipse = multivalued, dashed = derived), diamond = relationship, double diamond = identifying relationship, lines connect them and carry cardinality (1, N, M).

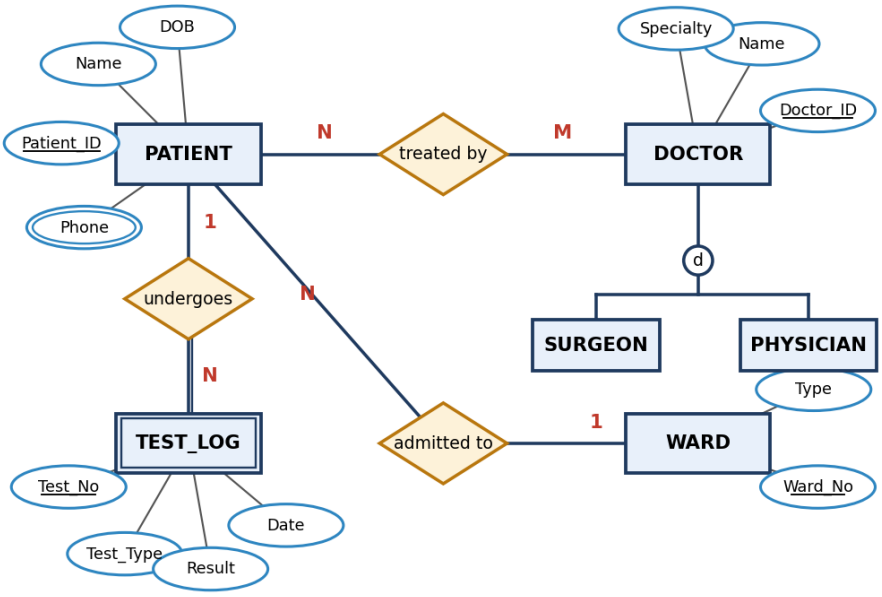
(a) University database (A Q2a)

Entities: STUDENT(Roll_No, Name, DOB, Age*), COURSE(Course_ID, Title, Credits), PROFESSOR(Prof_ID, Name, Phone, multivalued), DEPARTMENT(Dept_ID, Dept_Name, Location). **Relationships:** STUDENT enrolls in COURSE (M:N); PROFESSOR teaches COURSE (1:N, one professor teaches many courses); PROFESSOR works for DEPARTMENT (N:1); STUDENT belongs to DEPARTMENT (N:1). (* derived)



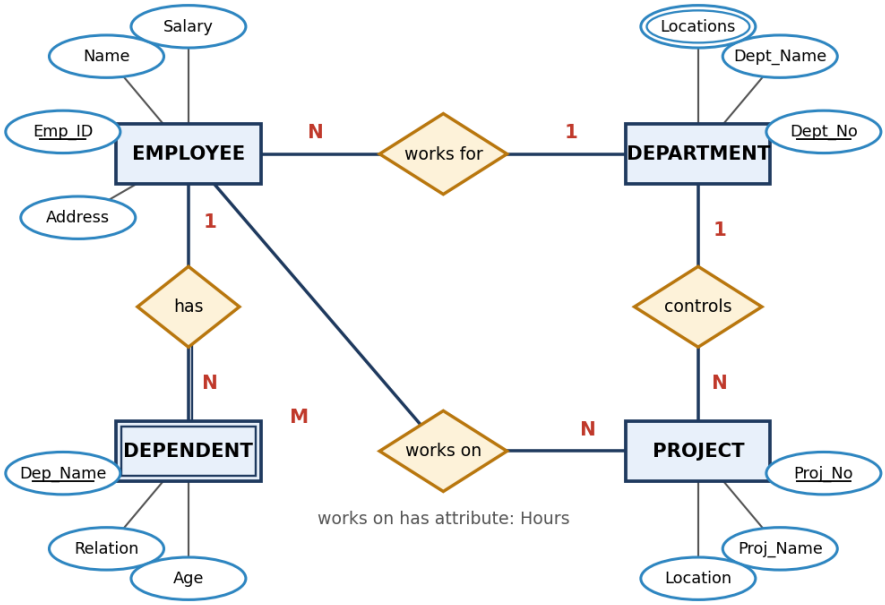
(b) Hospital EER diagram (B Q5a)

Entities: PATIENT, DOCTOR (specialized into SURGEON and PHYSICIAN, disjoint, marked 'd'), WARD, and TEST_LOG as a **weak entity** (each patient has a log of tests, identified through the patient). **Relationships:** DOCTOR treats PATIENT (M:N), PATIENT undergoes TEST_LOG (1:N, identifying), PATIENT admitted to WARD (N:1). Schema: see Topic 2.



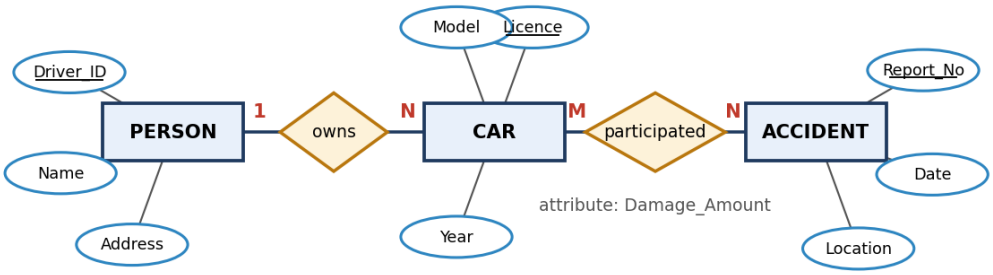
(c) Company database (C Q3a)

Entities: EMPLOYEE, DEPARTMENT (Locations multivalued), PROJECT, DEPENDENT (weak). **Relationships:** EMPLOYEE works for DEPARTMENT (N:1), DEPARTMENT controls PROJECT (1:N), EMPLOYEE works on PROJECT (M:N, with attribute Hours), EMPLOYEE has DEPENDENT (1:N, identifying). Schema: see Topic 2.



(d) Car-insurance company (D Q3ii-a)

Customers (persons) own one or more cars; each car has zero to many recorded accidents. A PERSON owns CARs (1:N, a car belongs to one owner); a CAR participates in ACCIDENTs (M:N, an accident may involve several insured cars) with the relationship attribute Damage_Amount. Zero accidents is allowed, so the CAR side of 'participated' is partial participation. Tables: see Topic 2.



9. DDL and DML Commands

Asked in: A Q1d · B Q1c · C Q1d · D Q1iii

Data Definition Language (DDL)

DDL commands **define and modify the structure** (schema) of database objects such as tables, views and indexes. DDL statements are auto-committed and cannot normally be rolled back. **Role in database management:** create the schema, change it as requirements evolve, remove objects, and (through constraints) define the rules data must satisfy; the DDL interpreter records definitions in the data dictionary.

Command	Purpose	Example
CREATE	Creates a table / object	CREATE TABLE Persons(id INT PRIMARY KEY, name VARCHAR(30));
ALTER	Add, modify or drop columns	ALTER TABLE Persons ADD Birthday DATE;
DROP	Removes the table and its structure	DROP TABLE Persons;
TRUNCATE	Removes all rows, keeps the structure	TRUNCATE TABLE Persons;
RENAME	Renames a table	RENAME TABLE Persons TO People;

Two DDL commands for an airline reservation system (B Q1c):

```
CREATE TABLE Flight (
  flight_no   VARCHAR(10) PRIMARY KEY,
  source      VARCHAR(30) NOT NULL,
  destination VARCHAR(30) NOT NULL,
  dep_time    TIMESTAMP,
  seats       INT CHECK (seats >= 0)
);
ALTER TABLE Flight ADD fare DECIMAL(8,2);
```

Data Manipulation Language (DML)

DML commands **store, modify, retrieve and delete the data** inside tables: SELECT, INSERT, UPDATE, DELETE (and MERGE). They can be rolled back until committed.

```
INSERT INTO Flight VALUES ('AI101','Mumbai','Delhi', SYSTIMESTAMP, 180, 5400);
UPDATE Flight SET fare = 5900 WHERE flight_no = 'AI101';
DELETE FROM Flight WHERE flight_no = 'AI101';
SELECT * FROM Flight WHERE destination = 'Delhi';
```

DDL vs DML

Basis	DDL	DML
Deals with	Structure (schema) of objects	Data stored in the tables
Commands	CREATE, ALTER, DROP, TRUNCATE, RENAME	SELECT, INSERT, UPDATE, DELETE
Rollback	Auto-commit; usually cannot be rolled back	Can be rolled back before COMMIT
WHERE clause	Not used	Used to select rows
Effect	Changes table definition	Changes table contents

TRUNCATE vs DROP vs DELETE (C Q1d)

Command	What it does	Type	Rollback / WHERE
DELETE	Removes selected (or all) rows, one by one; structure stays `DELETE FROM Student WHERE roll=3;`	DML	Can roll back; WHERE allowed
TRUNCATE	Removes **all** rows quickly; structure stays `TRUNCATE TABLE Student;`	DDL	Cannot roll back; no WHERE
DROP	Removes the **whole table**: data and structure `DROP TABLE Student;`	DDL	Cannot roll back

10. Transactions and ACID Properties

Asked in: B Q2f · C Q1f, Q2e · D Q1v

Transaction and the transaction process

A **transaction** is a set of logically related operations that accesses the database and is treated as a single unit of work. Basic operations: **Read(X)** copies X from the database into a main-memory buffer; **Write(X)** writes the buffer value back. **Commit** saves the work permanently; **Rollback** undoes it.

Example: transfer of Rs 800 from account X to account Y.

```
T:  R(X);  X = X - 800;  W(X);        -- debit X
    R(Y);  Y = Y + 800;  W(Y);        -- credit Y
    COMMIT;
```

If the system fails after W(X) but before W(Y), the money has left X but not reached Y, which is unacceptable. So the process is: the transaction starts (*active*), runs its last operation (*partially committed*), is made permanent (*committed*), or on any failure is rolled back (*failed, aborted*). See Topic 21 for the state diagram.

ACID properties

Illustration used throughout: A has Rs 600 and B has Rs 300; transaction T transfers Rs 100 from A to B (T1: A = A − 100, T2: B = B + 100).

Property	Meaning	Example / responsible component
Atomicity	All operations of a transaction take effect, or none do. No partial execution; on failure the changes are rolled back.	If T fails after T1 but before T2, A would lose Rs 100 which B never receives. Atomicity forces rollback. Handled by the transaction manager / recovery.
Consistency	A transaction moves the database from one consistent state to another; integrity constraints hold before and after.	Total before = 600 + 300 = 900; after = 500 + 400 = 900. Preserved only if T1 and T2 both run.
Isolation	Concurrent transactions do not interfere; the intermediate state of one is invisible to others, as if they ran serially.	While T uses A and B, another transaction cannot see A = 500, B = 300 (total 800). Enforced by the concurrency-control subsystem (locking).
Durability	Once committed, changes are permanent and survive system crashes or power failure.	After COMMIT, A = 500 and B = 400 remain even if the server crashes. Enforced by the recovery subsystem (logs, backups).

11. Generalization and Specialization

Asked in: B Q3a · C Q2a · D Q1iv

Generalization

Generalization is a **bottom-up** process in which two or more lower-level entities with common features are combined into a higher-level (super class) entity. It defines a general entity type from a set of specialized ones and minimizes differences by identifying common attributes. **Example:** TIGER, LION and ELEPHANT are generalized to ANIMAL; or CAR and TRUCK to VEHICLE(reg_no, owner, model) where the common attributes move to the super class.

Specialization

Specialization is a **top-down** process in which a higher-level entity is divided into sub groups (sub classes) based on distinguishing characteristics. Each sub class inherits the attributes of the super class and adds its own. **Example:** EMPLOYEE(emp_id, name, salary) is specialized into DEVELOPER(prog_language) and TESTER(testing_tool).

Differences

Generalization	Specialization
Bottom-up approach	Top-down approach
Combines several lower-level entities into one higher-level entity	Splits one higher-level entity into lower-level entities
Finds common attributes; reduces differences	Finds unique attributes; maximizes differences
Reverse of specialization	Reverse of generalization
Example: Tiger, Lion, Elephant → Animal	Example: Employee → Developer, Tester

Both lead to **super class / sub class** relationships (the 'is-a' relation) with **inheritance**, shown in an EER diagram with an ISA triangle or a circle (d = disjoint, o = overlapping). Related EER concepts are **aggregation** (a relationship treated as an entity) and **category/union**.

12. SQL Constraints (Integrity Constraints)

Asked in: B Q2c · C Q2c · D Q2A-ii

Constraints are rules applied to table columns to keep data valid, consistent and accurate; they prevent invalid data from being stored. Types of integrity: **entity** (PRIMARY KEY, NOT NULL), **referential** (FOREIGN KEY), **domain** (data type, CHECK, DEFAULT) and **uniqueness** (UNIQUE).

Constraint	Purpose	Example
NOT NULL	Column cannot hold NULL	Name VARCHAR(50) NOT NULL
UNIQUE	All values distinct (NULL allowed)	Email VARCHAR(100) UNIQUE
PRIMARY KEY	Uniquely identifies each row = NOT NULL + UNIQUE	Roll_No INT PRIMARY KEY
FOREIGN KEY	Value must exist in the referenced table's key; maintains referential integrity	FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
CHECK	Value must satisfy a condition	CHECK (Marks BETWEEN 0 AND 100)
DEFAULT	Value used when none is supplied	City VARCHAR(30) DEFAULT 'Mumbai'

Student database example with NOT NULL, UNIQUE and CHECK (B Q2c):

```
CREATE TABLE Department (
  Dept_ID   INT PRIMARY KEY,
  Dept_Name VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE Student (
  Roll_No  INT PRIMARY KEY,
  Name     VARCHAR(50) NOT NULL,                -- NOT NULL
  Email    VARCHAR(100) UNIQUE,                 -- UNIQUE
  Age      INT CHECK (Age >= 17),               -- CHECK
  Marks    INT CHECK (Marks BETWEEN 0 AND 100),
  City     VARCHAR(30) DEFAULT 'Mumbai',        -- DEFAULT
  Dept_ID  INT,
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)   -- referential integrity
);
```

Referential integrity rules: a foreign key value must match an existing primary key value in the parent table (or be NULL); when a parent key is deleted the matching child rows are deleted or set to NULL (ON DELETE CASCADE / SET NULL); when a parent key is updated the child values are updated. Inserting an employee into Dno 2 when department 2 does not exist is rejected.

13. Lock-Based Protocols, Two-Phase Locking and Deadlock

Asked in: A Q1f, Q3c · D Q3iii

The notes' Week 13 slides on lock-based protocols are pictures only, so this answer uses standard textbook content.

Lock-based protocols

Lock-based concurrency control makes a transaction obtain a **lock** on a data item before accessing it, so that conflicting accesses are serialized and the isolation property is kept. Lock modes:

- **Shared lock (S):** for reading; many transactions may hold S on the same item.
- **Exclusive lock (X):** for reading and writing; only one transaction may hold it.

Requested \ Held	S	X
S	Granted	Wait
X	Wait	Wait

A request is granted only if it is compatible with the locks already held; otherwise the transaction waits. Locks are released with an unlock operation.

Two-Phase Locking protocol (2PL) (D Q3iii)

2PL requires every transaction to acquire and release locks in two phases:

- **Growing phase:** the transaction may obtain locks but cannot release any.
- **Shrinking phase:** once it releases its first lock, it may release locks but cannot obtain new ones. The moment of the last lock acquisition is the **lock point**.

Example: T1 transfers money from A to B.

```
T1:  lock-X(A)   R(A)  A = A - 50  W(A)      -- growing
     lock-X(B)                                 -- growing  (lock point)
     R(B)  B = B + 50  W(B)
     unlock(A)   unlock(B)                     -- shrinking
```

Variants: Strict 2PL holds all exclusive locks until commit/abort (prevents cascading rollbacks); **Rigorous 2PL** holds all locks until commit; **Conservative 2PL** acquires all locks before starting (prevents deadlock).

Advantages	Disadvantages
Guarantees **conflict serializability** (the order of lock points gives the serial order)	Does **not** prevent **deadlock**
Simple and widely used; ensures isolation	Possible **cascading rollbacks** (unless strict/rigorous 2PL is used)
Works without knowing transactions in advance	Reduced concurrency, as locks are held for a long time; possible starvation

Deadlock and how locks help prevent it (A Q1f)

A **deadlock** occurs when two or more transactions each wait for a lock held by another, so none can proceed (T1 holds X(A) and waits for B; T2 holds X(B) and waits for A). Mechanisms:

- **Deadlock prevention:** *Conservative 2PL* (request all locks at the start); *ordering* of data items; timestamp-based schemes: **wait-die** (an older transaction may wait for a younger one, a younger one requesting a lock held by an older one is aborted) and **wound-wait** (an older transaction aborts a younger lock holder, a younger one waits).
- **Deadlock detection:** build a **wait-for graph** (edge Ti → Tj if Ti waits for a lock held by Tj); a **cycle means deadlock**. The graph is checked periodically.
- **Recovery from deadlock:** choose a **victim** (e.g. the transaction with least work done), roll it back (fully or partially), release its locks, and restart it; avoid starvation by not always picking the same victim.
- **Timeouts:** a transaction waiting longer than a set time is aborted (simple but may abort needlessly).

14. SQL Joins

Asked in: A Q2d · B Q3c

A join combines rows of two tables based on a related column. Sample tables: STUDENT(S_id, Name, C_id) = (1,Amit,C1), (2,Neha,C2), (3,Raj,NULL); COURSE(C_id, C_name) = (C1,DBMS), (C2,OS), (C3,CN).

```
-- INNER JOIN: only matching rows
SELECT S.Name, C.C_name FROM STUDENT S INNER JOIN COURSE C ON S.C_id = C.C_id;
-- LEFT JOIN: all students, matched course or NULL
SELECT S.Name, C.C_name FROM STUDENT S LEFT JOIN COURSE C ON S.C_id = C.C_id;
-- RIGHT JOIN: all courses, matched student or NULL
SELECT S.Name, C.C_name FROM STUDENT S RIGHT JOIN COURSE C ON S.C_id = C.C_id;
-- FULL OUTER JOIN: all rows of both tables
SELECT S.Name, C.C_name FROM STUDENT S FULL OUTER JOIN COURSE C ON S.C_id = C.C_id;
```

Join	Result on the sample data	Use it when
INNER	(Amit, DBMS), (Neha, OS)	you need only records that match in both tables, e.g. students with a course
LEFT	(Amit, DBMS), (Neha, OS), (Raj, NULL)	you need every row of the left table, matched or not, e.g. list all students including those with no course
RIGHT	(Amit, DBMS), (Neha, OS), (NULL, CN)	you need every row of the right table, e.g. all courses including ones with no students
FULL OUTER	all four rows: the three above plus (NULL, CN)	you need everything from both sides, e.g. a reconciliation report of unmatched records

(MySQL has no FULL OUTER JOIN; use LEFT JOIN UNION RIGHT JOIN.)

Natural join vs equi join (B Q3c)

Equi join joins on an explicit equality condition; both join columns appear in the result. **Natural join** automatically joins on all columns with the same name and shows the common column **once**.

```
-- Equi join: columns S.C_id and C.C_id both appear
SELECT * FROM STUDENT S, COURSE C WHERE S.C_id = C.C_id;
SELECT * FROM STUDENT S JOIN COURSE C ON S.C_id = C.C_id;
-- Natural join: C_id appears once
SELECT * FROM STUDENT NATURAL JOIN COURSE;
```

Both give (1, Amit, C1, DBMS) and (2, Neha, C2, OS). Theta join generalizes equi join to other comparison operators (<, >, <>).

15. Stored Procedures

Asked in: A Q1e · B Q4c

A **stored procedure** is a named PL/SQL subprogram (a block of SQL and procedural statements) that is **compiled and stored in the database** and executed on demand with parameters. It performs an action and does not return a value directly (unlike a function). Parts: declaration (optional), executable body (mandatory), exception handling (optional). Parameter modes: **IN** (passed in), **OUT** (returned), **IN OUT**.

```
CREATE [OR REPLACE] PROCEDURE procedure_name [(param [IN|OUT|IN OUT] type, ...)]
{IS | AS}
BEGIN
  <procedure_body>
END procedure_name;
/
EXECUTE procedure_name;      -- call;  DROP PROCEDURE procedure_name;  -- delete
```

Example (Employee database): give a percentage raise to an employee.

```
CREATE OR REPLACE PROCEDURE raise_salary (p_emp_id IN NUMBER, p_percent IN NUMBER) AS
BEGIN
  UPDATE employee
  SET    salary = salary + salary * p_percent / 100
  WHERE  emp_id = p_emp_id;
  COMMIT;
END raise_salary;
/
EXECUTE raise_salary(101, 10);
```

Advantages

- **Modularity and reusability:** write once, call from many programs.
- **Better performance:** stored in compiled form on the server; fewer round trips cut network traffic.
- **Security:** users can be given EXECUTE permission without access to the underlying tables.
- **Maintainability and consistency:** business rules live in one place; changing the procedure updates every caller.
- **Data integrity:** the same validation logic is applied for every application.

16. DBMS Architecture

Asked in: A Q1a · C Q1a

Architecture of a database system and its key components (A Q1a)

The structure of a DBMS has four parts: **users and interfaces** at the top, the **query processor** and the **storage manager** in the middle, and **disk storage** at the bottom.

- **Users:** naive users (use application programs, e.g. a bank teller), application programmers (write the programs), sophisticated users (write queries in a query language), specialized users, and the DBA (uses administration tools).
- **Query processor:** *DDL interpreter* (records definitions in the data dictionary), *DML compiler* (translates DML into an evaluation plan), *query optimizer* (picks the lowest-cost plan), *query evaluation engine* (executes the plan).
- **Storage manager:** *authorization and integrity manager* (checks constraints and access rights), *transaction manager* (keeps the database consistent despite failures and concurrency), *file manager* (allocates disk space), *buffer manager* (moves data between disk and main memory).
- **Disk storage:** *data files*, *data dictionary* (metadata, i.e. data about data) and *indices* (fast access).

Three-tier architecture (C Q1a)

The 3-tier architecture adds an **application server** between the client and the database server. The client never communicates with the database directly.



- **Client (presentation) tier:** user interface (browser or app) that sends requests.
- **Application (business logic) tier:** processes requests, applies business rules and validation, balances load, and talks to the database.
- **Database tier:** the DBMS and the stored data, which processes queries and transactions.

Advantages: better **scalability** (the database does not know individual clients; the application layer balances load), **security** (no direct client–database link), **data integrity** (checks on every request in the application layer). **Disadvantages:** more complex and harder to maintain because of the extra layer and additional communication points. Used by large web applications. For comparison, **1-tier** runs client, server and database on one machine (local development, learning SQL); **2-tier** is client–server with the client application talking directly to the database through ODBC/JDBC (multi-user, but less scalable and secure).

17. SQL vs PL/SQL

Asked in: B Q1e · C Q1e

Basis	SQL	PL/SQL
Full form	Structured Query Language	Procedural Language extension to SQL (Oracle)
Nature	Declarative: says *what* to do	Procedural: says *how* to do it, with logic
Unit of execution	A single statement at a time	A whole block of statements (DECLARE / BEGIN / EXCEPTION / END)
Programming constructs	None (no loops, conditions or variables)	Variables, IF, loops, cursors, exceptions, procedures, functions, triggers, packages
Error handling	Returns an error message	Exception-handling section (EXCEPTION WHEN ...)
Network traffic	Each statement sent separately	A block is sent at once, reducing traffic
Used for	Data definition, manipulation, retrieval	Building application logic inside the database; SQL statements can be embedded in it
Example	SELECT * FROM emp;	BEGIN FOR r IN (SELECT * FROM emp) LOOP ... END LOOP; END;

18. Types of Attributes in the ER Model

Asked in: B Q1a · C Q2b

An **attribute** is a property that describes an entity. Taking the entity STUDENT(Roll_No, Name, DOB, Age, Address, Phone_No):

Type	Meaning	Example	Notation
Simple (atomic)	Cannot be divided further	Roll_No, Name	Single ellipse
Composite	Made of several sub-attributes	Address = Street, City, State, Country	Ellipse with child ellipses
Single-valued	One value per entity	DOB	Single ellipse
Multivalued	More than one value for an entity	Phone_No (many phones)	Double ellipse
Derived	Computed from other attributes	Age (from DOB)	Dashed ellipse
Key	Uniquely identifies each entity	Roll_No	Underlined name

Applying it to a **library database** (C Q2b): BOOK(ISBN key, Title simple, Authors multivalued, Publisher composite = Name + City, Age_of_book derived from Year_of_publication).

19. SQL Aggregate Functions

Asked in: B Q3f · C Q3c(v)

An **aggregate function** performs a calculation on a set of rows of a column and returns a **single value**. NULLs are ignored (except COUNT(*)). For the employee database EMPLOYEE(emp_id, name, dept, salary):

```
SELECT COUNT(*)         FROM employee;                  -- number of employees
SELECT COUNT(DISTINCT dept) FROM employee;              -- number of different departments
SELECT SUM(salary)      FROM employee;                  -- total salary
SELECT AVG(salary)      FROM employee;                  -- average salary
SELECT MAX(salary)      FROM employee;                  -- highest salary
SELECT MIN(salary) AS lowest FROM employee;             -- lowest salary (alias with AS)

-- with GROUP BY and HAVING
SELECT dept, COUNT(*), AVG(salary), MAX(salary)
FROM   employee
GROUP  BY dept
HAVING AVG(salary) > 30000;
```

Function	Description
COUNT	Counts rows; COUNT(*) counts all rows including duplicates and NULLs, COUNT(col) skips NULLs
SUM	Sum of a numeric column
AVG	Average of non-NULL values
MAX / MIN	Largest / smallest value

WHERE filters rows *before* grouping; HAVING filters groups *after* aggregation.

20. Recovery System

Asked in: A Q2f, Q3c

Recovery restores the database to a consistent state after a failure, preserving **atomicity and durability**. **Scenario (A Q2f):** the system crashes after a transaction reached the *partially committed* state (its last statement ran, but its updates may still be only in memory buffers and the commit may not be fully recorded on disk).

Log-based recovery with checkpoints: steps after the crash

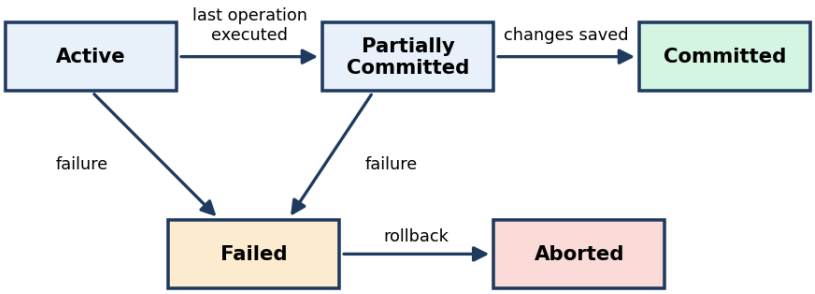
- **Write-ahead logging (before the crash):** every update is recorded in the log as <T, item, old value, new value>, and the log is forced to stable storage *before* data pages and before the <T commit> record.
- **Restart and scan the log:** start from the most recent **checkpoint** record to avoid scanning the whole log.
- **Classify transactions:** those with both <T start> and <T commit> in the log are **committed**; those with <T start> but no <T commit> are **incomplete**. A partially committed transaction whose commit record never reached the disk is incomplete.
- **UNDO** every incomplete transaction: restore the *old values* from the log, scanning backwards, and write an <T abort> record. This removes partial effects.
- **REDO** every committed transaction: re-apply the *new values* from the log, scanning forwards, in case its updates were not yet written to disk. This ensures durability.
- **Resume:** the database is now consistent, with all committed work present and no partial work; normal processing continues (aborted transactions may be restarted).

Alternative: shadow paging (from the notes)

Shadow paging recovers without a log, using two page tables. At the start the **shadow page table** (copy of the current table) is saved on disk and never changed. Updates create **new copies of pages** referenced only by the **current page table**. **Commit:** flush modified pages, write the current page table to disk, and atomically switch the fixed pointer so that the current table becomes the shadow table. **Crash before commit:** discard the current page table and free the new pages; the shadow table still points to the old consistent pages, so no undo or redo is needed. **Advantages:** no log, fast recovery. **Disadvantages:** commit overhead, fragmentation of pages, garbage collection, hard to extend to concurrent transactions.

21. Transaction States

Asked in: B Q3d



State	Description
Active	Initial state; the transaction is executing its read/write operations. Changes are not yet permanent.
Partially committed	The final operation has been executed, but the data has not yet been saved permanently.
Committed	All operations succeeded and the changes are permanently stored in the database.
Failed	A check by the recovery system fails, or normal execution can no longer continue (e.g. hardware or logic error).
Aborted	The transaction is rolled back and the database is returned to its state before the transaction. The system then either **restarts** the transaction or **kills** it.

Transitions: Active → Partially committed (last operation done) → Committed (changes saved); Active or Partially committed → Failed (failure) → Aborted (rollback).

22. Cursors and Functions in PL/SQL

Asked in: B Q2e

Functions

A **function** is a named PL/SQL subprogram, like a procedure, that **must return a single value** using RETURN and can be used inside SQL expressions.

```
CREATE OR REPLACE FUNCTION totalCustomers RETURN NUMBER IS
  total NUMBER(2) := 0;
BEGIN
  SELECT COUNT(*) INTO total FROM customers;
  RETURN total;
END;
/
DECLARE c NUMBER(2);
BEGIN
  c := totalCustomers();
  dbms_output.put_line('Total no. of Customers: ' || c);
END;
/
```

A function with parameters, e.g. findMax(x IN NUMBER, y IN NUMBER) RETURN NUMBER, returns the larger value (findMax(23,45) gives 45); a function can even call itself, as in a recursive factorial (fact(6) = 720).

Cursors

A **cursor** is a pointer to the private memory area (context area) that holds the result set of a SQL statement, so that PL/SQL can process a multi-row result **row by row**.

- **Implicit cursor:** created automatically by Oracle for every DML statement and single-row SELECT INTO. Attributes: SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT.
- **Explicit cursor:** declared by the programmer for multi-row queries. Four steps: **DECLARE** (define the query), **OPEN** (execute it), **FETCH** (read a row into variables), **CLOSE** (release memory). Attributes: %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN.

```
DECLARE
  CURSOR c_emp IS SELECT emp_name, salary FROM employee WHERE salary > 30000;
  v_name employee.emp_name%TYPE;
  v_sal  employee.salary%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_name, v_sal;
    EXIT WHEN c_emp%NOTFOUND;
    dbms_output.put_line(v_name || ' earns ' || v_sal);
  END LOOP;
  CLOSE c_emp;
END;
/
```

23. Total and Partial Participation

Asked in: B Q2a

Participation constraints specify whether every entity of an entity set must take part in a relationship.

- **Total participation** (shown with a **double line**): every entity in the set participates in at least one relationship instance.
- **Partial participation** (single line): only some entities of the set participate.

College database examples:

- STUDENT *enrolls in* COURSE: every student must be enrolled in at least one course, so STUDENT has **total** participation. A course may currently have no students, so COURSE has **partial** participation.
- PROFESSOR *heads* DEPARTMENT: every department must have a head (DEPARTMENT total), but only some professors are heads (PROFESSOR partial).
- STUDENT *belongs to* DEPARTMENT: every student belongs to a department (total); a new department may have no students yet (partial).

Why it matters in mapping: total participation on the N side of a 1:N or on one side of a 1:1 relationship allows the relationship to be merged into that entity's table, and the foreign key can be declared NOT NULL; with partial participation on both sides of a 1:1 relationship, merging would create NULLs, so separate tables are kept.

24. Super Key and Candidate Key

Asked in: B Q1b

Consider Student(roll, name, gender, age, address, class, section).

- **Super key:** any set of attributes that uniquely identifies each tuple; it may contain extra attributes. Examples: (roll), (roll, name), (class, section, roll, gender), (name, address), and all attributes together. The number of attributes is unrestricted.
- **Candidate key:** a **minimal** super key, i.e. no proper subset of it is a super key. Examples: (class, section, roll) and (name, address), assuming no two students share a name and address. (roll, name) is a super key but not a candidate key because roll alone suffices (if roll is unique).
- **Primary key:** the candidate key chosen by the designer; it cannot be NULL. **Alternate key:** candidate keys not chosen as primary. **Foreign key:** an attribute referencing the primary key of another table. **Composite key:** a key of two or more attributes.

Relationship: every candidate key is a super key, but not every super key is a candidate key: Candidate keys $\subset$ Super keys, Primary key $\in$ Candidate keys.

STUD_NO	STUD_PHONE	STUD_NAME	STUD_AGE
1	98201	Amit	19
2	99999	Neha	21

Here STUD_NO and STUD_PHONE are both candidate keys; STUD_NO is chosen as the primary key and STUD_PHONE is the alternate key. (STUD_NO, STUD_AGE) is a super key but not a candidate key.

25. Database Administrator (DBA) Responsibilities

Asked in: D Q1i

A **DBA** is the professional who manages, maintains and secures the DBMS, ensuring data integrity, availability and performance. **Five main responsibilities and the problems if each is neglected:**

Responsibility	Problem if not discharged properly
Database design and implementation: define schema, tables, relationships	Redundancy, anomalies and inconsistent data; the system does not meet users' needs
Security management: access control, authentication, encryption, privileges (GRANT/REVOKE)	Unauthorized access, data theft or tampering, privacy violations
Backup and recovery: regular backups and tested recovery procedures	Permanent data loss after a crash, long downtime, loss of business continuity
Performance monitoring and tuning: indexes, query optimization, configuration	Slow queries, bottlenecks, poor response times and unhappy users
Troubleshooting, migration, capacity planning and documentation	Unresolved failures, running out of storage, failed upgrades, and no record of how the system works

Other duties: integrity-constraint definition, user training and support, upgrades and patches, and coordinating with developers.